DATASTOCK · REGISTRO DE PRUEBAS

UT2 · Acceso a bases de datos con JDBC | 2.º DAM | Curso 2026–2027

Alumno/a	Alfonso Sevillano Martinez
Repositorio GitHub (URL)	https://github.com/alfonsosemar/DataStock
Commit / etiqueta de entrega	Etiqueta UT2-v1.0
Fecha de ejecución	09/11/2026
Entorno	macOS · JDK 25 (compilado con release 21) · Maven · MySQL 9.6.0 · Connector/J 9.1.0 · sqlite-jdbc 3.46.1.0
Base de datos de pruebas	datastock_pruebas, creada con el script DataStock_BD_MySQL.sql y vacía al empezar. Se selecciona con la variable JDBC_URL para no modificar la base de datos datastock.

1. Resumen de pruebas

Las pruebas se ejecutaron en orden sobre la misma base de datos, por lo que cada una parte del estado que deja la anterior. Los datos de entrada son los que se escribieron en la consola.

ID prueba	RF	Entrada / datos	Esperado	Obtenido	Estado
CP-JDBC-01	RF-JDBC-01	Menú 7 > opción 1.	Mensaje de conexión, nombre de la BD, versión del gestor, número de productos y procedimiento disponible.	Se muestran la BD datastock_pruebas, MySQL 9.6.0, Connector/J 9.1.0, 0 productos y el procedimiento disponible.	**SUPERADA**
CP-JDBC-02	RF-JDBC-01	JDBC_PASSWORD=claveIncorrecta; menú 7 > opción 1.	Mensaje controlado, sin la contraseña, y el menú sigue funcionando.	Mensaje de error claro (código 1045). La contraseña no aparece en la salida. El menú MySQL se vuelve a mostrar.	**SUPERADA**
CP-JDBC-03	RF-JDBC-02	Categorías 1 "Periféricos" y 2 "Monitores". Proveedor 1 "TecnoSur", ventas@tecnosur.es.	Altas confirmadas con [OK] y registros visibles en los listados.	Las tres altas devuelven [OK] y los listados muestran los registros ordenados por id.	**SUPERADA**
CP-JDBC-04	RF-JDBC-02	Categoría id 1; categoría 5 "Periféricos"; baja categoría 1; proveedor 2 con email ventas@tecnosur.es.	Cuatro errores controlados; no se borra nada y el programa no termina.	Se muestran los cuatro errores y la categoría 1 y sus productos se conservan.	**SUPERADA**
CP-JDBC-05	RF-JDBC-03	TEC-001, "Teclado mecánico", 49.90, stock 8, cat. 1, prov. 1. También ALT-004, "Altavoces USB", 25.50, stock 3, cat. 2, prov. 1.	Producto guardado y ficha con todos los datos, incluidos los nombres de categoría y proveedor.	Se guarda y la ficha y la tabla producto coinciden.	**SUPERADA**
CP-JDBC-06	RF-JDBC-03	Nombre "Teclado mecánico RGB", precio 54.90; Intro en categoría y proveedor.	Producto actualizado; stock sigue en 8.	Actualizado; MySQL muestra el nuevo nombre y precio con stock 8.	**SUPERADA**
CP-JDBC-07	RF-JDBC-03	Alta TEC-001; alta RAT-002 con categoría 99 (después Intro para cancelar).	Ambos casos rechazados; no se inserta nada.	Errores controlados; siguen existiendo solo 2 productos en MySQL.	**SUPERADA**
CP-JDBC-08	RF-JDBC-03	Alta y baja de BOR-001; baja de TEC-001.	BOR-001 eliminado; TEC-001 rechazado por la FK y el historial se conserva.	Correcto: BOR-001 desaparece y TEC-001 y sus 2 movimientos siguen en MySQL.	**SUPERADA**
CP-JDBC-09	RF-JDBC-04	Menú 7 > 5 > 1.	Tabla ordenada por código con los nombres obtenidos con JOIN.	Se muestran los 2 productos ordenados, con "Monitores"/"Periféricos" y "TecnoSur".	**SUPERADA**
CP-JDBC-10	RF-JDBC-04	Nombre "tecl"; nombre "zzz"; código NOE-999.	Encuentra TEC-001; los otros dos casos muestran un aviso y vuelven al menú.	Correcto (la búsqueda no distingue mayúsculas).	**SUPERADA**
CP-JDBC-11	RF-JDBC-04	Umbral 5.	Solo aparece ALT-004.	Solo aparece ALT-004.	**SUPERADA**

ID prueba	RF	Entrada / datos	Esperado	Obtenido	Estado
CP-JDBC-12	RF-JDBC-05	TEC-001, ENTRADA, 5.	Stock 8 -> 13 y un movimiento nuevo.	Stock 8 -> 13; movimiento 1 registrado en movimiento_stock.	**SUPERADA**
CP-JDBC-13	RF-JDBC-05	TEC-001, SALIDA, 4.	Stock 13 -> 9.	Stock 13 -> 9; movimiento 2 registrado.	**SUPERADA**
CP-JDBC-14	RF-JDBC-05	TEC-001, SALIDA, 20.	Error; stock sigue en 9 y siguen existiendo 2 movimientos.	Error del procedimiento (SIGNAL 45000) traducido; stock 9 y 2 movimientos.	**SUPERADA**
CP-JDBC-15	RF-JDBC-06	Menú 7 > 7 > 1; después opción 2 con TEC-001.	Movimientos con id, fecha, tipo, cantidad, stock anterior y posterior, en orden.	Correcto en ambos casos.	**SUPERADA**
CP-JDBC-16	RF-JDBC-06	Menú 7 > 7 > 2 con ALT-004.	Aviso de que no tiene movimientos.	Aviso mostrado; no se modifica nada.	**SUPERADA**
CP-JDBC-17	RF-JDBC-07	TEC-001 (ya existe), RAT-002 (válido), MON-003 (proveedor 3 inexistente).	Insertado 1, omitido 1, rechazado 1; memoria sin cambios.	Resumen 3 / 1 / 1 / 1. En MySQL aparece RAT-002. En memoria siguen 3 productos.	**SUPERADA**
CP-JDBC-18	RF-JDBC-07	Mientras el programa espera la confirmación final, se inserta NEW-002 desde fuera (MySQL).	Error de clave duplicada en NEW-002, rollback y NEW-001 no guardado.	Rollback realizado. En MySQL solo está el NEW-002 insertado desde fuera; NEW-001 no aparece.	**SUPERADA**
CP-JDBC-19	RF-JDBC-08	Menú 7 > 9. Categorías 9001 y 9002.	Las dos categorías quedan guardadas.	Commit correcto; el SELECT encuentra 9001 y 9002.	**SUPERADA**
CP-JDBC-20	RF-JDBC-08	Escenario B de la misma demostración: categoría 9003 insertada dos veces.	Error capturado, rollback y la 9003 no existe.	Correcto: la tabla categoria no contiene la 9003. autoCommit restaurado.	**SUPERADA**
CP-SQLITE-01	Apartado 7	Ejecutar la clase DemoSQLite.	Se crea el fichero, 2 insertados y se listan SQL-001 y SQL-002.	Correcto.	**SUPERADA**
CP-SQLITE-02	Apartado 7	Ejecutar DemoSQLite otra vez.	0 insertados, 2 ya existentes y solo 2 filas.	Correcto (INSERT OR IGNORE).	**SUPERADA**
CP-SQLITE-03	Apartado 7	Listar datos/sqlite y comprobar con lsof que ningún proceso tiene el fichero abierto.	El fichero existe y no está abierto por ningún proceso.	El fichero existe (12288 bytes) y lsof no muestra procesos. El código usa try-with-resources.	**SUPERADA**

Estados posibles: SUPERADA / NO SUPERADA / NO EJECUTADA.

2. Fichas detalladas de las pruebas

La evidencia es la salida de consola copiada de la ejecución real, recortada a las líneas importantes, junto con la comprobación hecha en MySQL cuando corresponde.

CP-JDBC-01 · RF-JDBC-01 · Conexión correcta

Objetivo	Comprobar que se conecta a MySQL y se muestran los datos de la base de datos.
Precondiciones	MySQL arrancado. Esquema creado con DataStock_BD_MySQL.sql. Credenciales en config/datastock.properties.
Datos de entrada	Menú 7 > opción 1.
Pasos ejecutados	1. Abrir el menú 7. 2. Elegir la opción 1.
Resultado esperado	Mensaje de conexión, nombre de la BD, versión del gestor, número de productos y procedimiento disponible.
Resultado obtenido	Se muestran la BD datastock_pruebas, MySQL 9.6.0, Connector/J 9.1.0, 0 productos y el procedimiento disponible.
Estado	**SUPERADA**

Evidencia	`[OK] Conexión establecida correctamente.`
	`Base de datos: datastock_pruebas`
	`Gestor: MySQL`
	`Versión: 9.6.0`
	`Driver JDBC: MySQL Connector/J mysql-connector-j-9.1.0`
	`Productos almacenados: 0`
	`Procedimiento registrar_movimiento: disponible`

CP-JDBC-02 · RF-JDBC-01 · Contraseña incorrecta

Objetivo	Comprobar que un error de credenciales se informa sin mostrar la contraseña y la aplicación continúa.
Precondiciones	Variable de entorno JDBC_PASSWORD con una contraseña incorrecta.
Datos de entrada	JDBC_PASSWORD=claveIncorrecta; menú 7 > opción 1.
Pasos ejecutados	1. Ejecutar con la contraseña errónea. 2. Menú 7 > 1. 3. Volver al menú y salir.
Resultado esperado	Mensaje controlado, sin la contraseña, y el menú sigue funcionando.
Resultado obtenido	Mensaje de error claro (código 1045). La contraseña no aparece en la salida. El menú MySQL se vuelve a mostrar.
Estado	**SUPERADA**

Evidencia	`Selecciona una opción: 1`
	`[ERROR] No se ha podido conectar. MySQL ha rechazado el usuario o la contraseña.`
	`==========`
	`GESTIÓN MYSQL - UT2`
	`==========`

CP-JDBC-03 · RF-JDBC-02 · Alta y listado de categoría y proveedor

Objetivo	Dar de alta categorías y un proveedor y comprobar que aparecen en el listado.
Precondiciones	Base de datos vacía.
Datos de entrada	Categorías 1 "Periféricos" y 2 "Monitores". Proveedor 1 "TecnoSur", ventas@tecnosur.es.
Pasos ejecutados	1. Menú 7 > 2 > 1 (dos altas). 2. Opción 2 (listar). 3. Menú 7 > 3 > 1 (alta proveedor). 4. Opción 2.
Resultado esperado	Altas confirmadas con [OK] y registros visibles en los listados.
Resultado obtenido	Las tres altas devuelven [OK] y los listados muestran los registros ordenados por id.
Estado	**SUPERADA**

Evidencia	`[OK] Categoría registrada correctamente.`
	`[OK] Categoría registrada correctamente.`
	`ID: 1 | Nombre: Periféricos`
	`ID: 2 | Nombre: Monitores`
	`[OK] Proveedor registrado correctamente.`
	`ID: 1 | Nombre: TecnoSur | Email: ventas@tecnosur.es`

CP-JDBC-04 · RF-JDBC-02 · Duplicados y baja con productos asociados

Objetivo	Comprobar que se rechazan id, nombre y email duplicados y que no se borra una categoría con productos.
Precondiciones	Categoría 1 con productos (TEC-001). Proveedor 1 con email ventas@tecnosur.es.
Datos de entrada	Categoría id 1; categoría 5 "Periféricos"; baja categoría 1; proveedor 2 con email ventas@tecnosur.es.
Pasos ejecutados	1. Alta categoría con id 1. 2. Alta categoría 5 con nombre repetido. 3. Baja categoría 1. 4. Alta proveedor con email repetido.
Resultado esperado	Cuatro errores controlados; no se borra nada y el programa no termina.
Resultado obtenido	Se muestran los cuatro errores y la categoría 1 y sus productos se conservan.
Estado	**SUPERADA**

Evidencia	`[ERROR] Ya existe una categoría con ID 1.`
	`[ERROR] Ya existe otra categoría llamada "Periféricos".`
	`[ERROR] No se puede eliminar la categoría 1: tiene productos asociados.`
	`Se conservan todos los registros.`
	`[ERROR] El email ventas@tecnosur.es ya está registrado en otro proveedor.`

CP-JDBC-05 · RF-JDBC-03 · Insertar y recuperar producto

Objetivo	Insertar un producto con todos sus datos y recuperarlo por código.
Precondiciones	Categorías 1 y 2 y proveedor 1 existentes.
Datos de entrada	TEC-001, "Teclado mecánico", 49.90, stock 8, cat. 1, prov. 1. También ALT-004, "Altavoces USB", 25.50, stock 3, cat. 2, prov. 1.
Pasos ejecutados	1. Menú 7 > 4 > 1 (alta). 2. Opción 2 (consultar TEC-001). 3. Comprobar en MySQL.
Resultado esperado	Producto guardado y ficha con todos los datos, incluidos los nombres de categoría y proveedor.
Resultado obtenido	Se guarda y la ficha y la tabla producto coinciden.
Estado	**SUPERADA**

Evidencia	`[OK] Producto TEC-001 guardado correctamente.`
	`[OK] Producto ALT-004 guardado correctamente.`
	`mysql> SELECT * FROM producto;`
	`| ALT-004 | Altavoces USB | 25.50 | 3 | 2 | 1 |`
	`| TEC-001 | Teclado mecánico | 49.90 | 8 | 1 | 1 |`

CP-JDBC-06 · RF-JDBC-03 · Modificar nombre y precio

Objetivo	Modificar el nombre y el precio y comprobar el cambio en MySQL (el stock no cambia).
Precondiciones	TEC-001 existente con precio 49.90 y stock 8.
Datos de entrada	Nombre "Teclado mecánico RGB", precio 54.90; Intro en categoría y proveedor.
Pasos ejecutados	1. Menú 7 > 4 > 3. 2. Introducir TEC-001 y los nuevos datos. 3. Confirmar con S. 4. SELECT en MySQL.
Resultado esperado	Producto actualizado; stock sigue en 8.
Resultado obtenido	Actualizado; MySQL muestra el nuevo nombre y precio con stock 8.
Estado	**SUPERADA**

Evidencia	`Nuevos datos -> Nombre: Teclado mecánico RGB | Precio: 54.90 € | Categoría: 1 | Proveedor: 1`
	`[AVISO] ¿Guardar los cambios? (S/N): S`
	`[OK] Producto TEC-001 actualizado.`
	`mysql> SELECT codigo,nombre,precio,stock FROM producto WHERE codigo='TEC-001';`
	`| TEC-001 | Teclado mecánico RGB | 54.90 | 8 |`

CP-JDBC-07 · RF-JDBC-03 · Código duplicado y categoría inexistente

Objetivo	Comprobar que no se duplica un código y que no se acepta una categoría que no existe.
Precondiciones	TEC-001 existente. No existe la categoría 99.
Datos de entrada	Alta TEC-001; alta RAT-002 con categoría 99 (después Intro para cancelar).
Pasos ejecutados	1. Alta con código TEC-001. 2. Alta RAT-002 con categoría 99. 3. Cancelar.
Resultado esperado	Ambos casos rechazados; no se inserta nada.
Resultado obtenido	Errores controlados; siguen existiendo solo 2 productos en MySQL.
Estado	**SUPERADA**

Evidencia	`[ERROR] El producto TEC-001 ya existe en MySQL. No se sobrescribe.`
	`ID de categoría (Intro para cancelar): 99`
	`[ERROR] No existe la categoría con ID 99. Dala de alta antes o indica otra.`
	`[AVISO] Alta cancelada. No se ha insertado nada.`
	`mysql> SELECT COUNT(*) FROM producto;  -> 2`

CP-JDBC-08 · RF-JDBC-03 · Baja permitida y baja bloqueada

Objetivo	Borrar un producto sin movimientos y comprobar que no se puede borrar uno con movimientos.
Precondiciones	TEC-001 con 2 movimientos registrados (CP-JDBC-12 y 13).
Datos de entrada	Alta y baja de BOR-001; baja de TEC-001.
Pasos ejecutados	1. Alta BOR-001. 2. Baja BOR-001 confirmando. 3. Baja TEC-001 confirmando.
Resultado esperado	BOR-001 eliminado; TEC-001 rechazado por la FK y el historial se conserva.
Resultado obtenido	Correcto: BOR-001 desaparece y TEC-001 y sus 2 movimientos siguen en MySQL.
Estado	**SUPERADA**

Evidencia	`[OK] Producto BOR-001 guardado correctamente.`
	`[AVISO] ¿Confirmas la eliminación? (S/N): S`
	`[OK] Producto BOR-001 eliminado.`
	`[AVISO] ¿Confirmas la eliminación? (S/N): S`
	`[ERROR] No se puede eliminar TEC-001: tiene movimientos de stock registrados.`
	`Se conserva el historial.`
	`mysql> SELECT COUNT(*) FROM movimiento_stock; -> 2`

CP-JDBC-09 · RF-JDBC-04 · Consulta con JOIN y orden

Objetivo	Listar productos ordenados por código con el nombre de categoría y proveedor.
Precondiciones	ALT-004 y TEC-001 en MySQL.
Datos de entrada	Menú 7 > 5 > 1.
Pasos ejecutados	1. Menú 7 > 5. 2. Opción 1.
Resultado esperado	Tabla ordenada por código con los nombres obtenidos con JOIN.
Resultado obtenido	Se muestran los 2 productos ordenados, con "Monitores"/"Periféricos" y "TecnoSur".
Estado	**SUPERADA**

Evidencia	`CÓDIGO NOMBRE PRECIO € STOCK CATEGORÍA PROVEEDOR`
	`ALT-004 Altavoces USB 25.50 3 Monitores TecnoSur`
	`TEC-001 Teclado mecánico RGB 54.90 8 Periféricos TecnoSur`
	`Total de productos encontrados: 2`

CP-JDBC-10 · RF-JDBC-04 · Búsqueda parcial y sin resultados

Objetivo	Buscar por fragmento de nombre con LIKE y comprobar los mensajes cuando no hay resultados.
Precondiciones	Productos de CP-JDBC-09.
Datos de entrada	Nombre "tecl"; nombre "zzz"; código NOE-999.
Pasos ejecutados	1. Menú 7 > 5 > 3 con "tecl". 2. Con "zzz". 3. Opción 2 con NOE-999.
Resultado esperado	Encuentra TEC-001; los otros dos casos muestran un aviso y vuelven al menú.
Resultado obtenido	Correcto (la búsqueda no distingue mayúsculas).
Estado	**SUPERADA**

Evidencia	`Texto del nombre: tecl`
	`TEC-001 Teclado mecánico RGB 54.90 8 Periféricos TecnoSur`
	`Total de productos encontrados: 1`
	`[AVISO] No hay productos cuyo nombre contenga "zzz".`
	`[AVISO] No existe ningún producto con el código NOE-999.`

CP-JDBC-11 · RF-JDBC-04 · Umbral de stock

Objetivo	Mostrar los productos con stock inferior a un umbral.
Precondiciones	ALT-004 stock 3; TEC-001 stock 8.
Datos de entrada	Umbral 5.
Pasos ejecutados	1. Menú 7 > 5 > 4. 2. Umbral 5.
Resultado esperado	Solo aparece ALT-004.
Resultado obtenido	Solo aparece ALT-004.
Estado	**SUPERADA**

Evidencia	`Umbral (entero mayor o igual que 0): 5`
	`ALT-004 Altavoces USB 25.50 3 Monitores TecnoSur`
	`Total de productos encontrados: 1`

CP-JDBC-12 · RF-JDBC-05 · Entrada de 5 unidades

Objetivo	Registrar una entrada con el procedimiento y comprobar producto e historial.
Precondiciones	TEC-001 con stock 8.
Datos de entrada	TEC-001, ENTRADA, 5.
Pasos ejecutados	1. Menú 7 > 6 > 1. 2. TEC-001, cantidad 5, confirmar. 3. Comprobar ambas tablas.
Resultado esperado	Stock 8 -> 13 y un movimiento nuevo.
Resultado obtenido	Stock 8 -> 13; movimiento 1 registrado en movimiento_stock.
Estado	**SUPERADA**

Evidencia	`[OK] Movimiento 1 registrado: ENTRADA de 5 unidades.`
	`Stock anterior: 8 -> stock posterior: 13`
	`mysql> SELECT id,producto_codigo,tipo,cantidad,stock_anterior,stock_posterior`
	`FROM movimiento_stock;`
	`| 1 | TEC-001 | ENTRADA | 5 | 8 | 13 |`

CP-JDBC-13 · RF-JDBC-05 · Salida válida

Objetivo	Registrar una salida y comprobar el cálculo del stock.
Precondiciones	TEC-001 con stock 13.
Datos de entrada	TEC-001, SALIDA, 4.
Pasos ejecutados	1. Menú 7 > 6 > 2. 2. TEC-001, cantidad 4, confirmar.
Resultado esperado	Stock 13 -> 9.
Resultado obtenido	Stock 13 -> 9; movimiento 2 registrado.
Estado	**SUPERADA**

Evidencia	`[OK] Movimiento 2 registrado: SALIDA de 4 unidades.`
	`Stock anterior: 13 -> stock posterior: 9`
	`mysql> ... | 2 | TEC-001 | SALIDA | 4 | 13 | 9 |`

CP-JDBC-14 · RF-JDBC-05 · Salida superior al stock

Objetivo	Comprobar que una salida mayor que el stock se rechaza sin crear movimiento.
Precondiciones	TEC-001 con stock 9 y 2 movimientos.
Datos de entrada	TEC-001, SALIDA, 20.
Pasos ejecutados	1. Menú 7 > 6 > 2. 2. TEC-001, cantidad 20, confirmar. 3. Comprobar en MySQL.
Resultado esperado	Error; stock sigue en 9 y siguen existiendo 2 movimientos.
Resultado obtenido	Error del procedimiento (SIGNAL 45000) traducido; stock 9 y 2 movimientos.
Estado	**SUPERADA**

Evidencia	`[ERROR] Stock insuficiente. El stock de TEC-001 permanece en 9.`
	`mysql> SELECT codigo,stock FROM producto WHERE codigo='TEC-001'; -> 9`
	`mysql> SELECT COUNT(*) FROM movimiento_stock; -> 2`

CP-JDBC-15 · RF-JDBC-06 · Historial ordenado y filtrado

Objetivo	Consultar todos los movimientos y los de un producto, ordenados por fecha e id.
Precondiciones	2 movimientos de TEC-001.
Datos de entrada	Menú 7 > 7 > 1; después opción 2 con TEC-001.
Pasos ejecutados	1. Historial completo. 2. Historial de TEC-001.
Resultado esperado	Movimientos con id, fecha, tipo, cantidad, stock anterior y posterior, en orden.
Resultado obtenido	Correcto en ambos casos.
Estado	**SUPERADA**

Evidencia	`ID FECHA PRODUCTO TIPO CANTIDAD ANTERIOR POSTERIOR`
	`1 2026-11-09 17:59:51 TEC-001 ENTRADA 5 8 13`
	`2 2026-11-09 17:59:51 TEC-001 SALIDA 4 13 9`
	`Total de movimientos: 2`

CP-JDBC-16 · RF-JDBC-06 · Producto sin movimientos

Objetivo	Comprobar el mensaje para un producto que existe pero no tiene movimientos.
Precondiciones	ALT-004 sin movimientos.
Datos de entrada	Menú 7 > 7 > 2 con ALT-004.
Pasos ejecutados	1. Historial de un producto. 2. ALT-004.
Resultado esperado	Aviso de que no tiene movimientos.
Resultado obtenido	Aviso mostrado; no se modifica nada.
Estado	**SUPERADA**
Evidencia	`Código del producto (Intro para cancelar): ALT-004` `[AVISO] Sin movimientos registrados para ALT-004.`

CP-JDBC-17 · RF-JDBC-07 · Importación con válidos, duplicados y referencias inexistentes

Objetivo	Importar el inventario en memoria y comprobar el resumen.
Precondiciones	En MySQL: TEC-001; no existe el proveedor 3. Memoria: datos/ejemplos/catalogo_ut2.json.
Datos de entrada	TEC-001 (ya existe), RAT-002 (válido), MON-003 (proveedor 3 inexistente).
Pasos ejecutados	1. Menú 2: importar catalogo_ut2.json. 2. Menú 7 > 8. 3. Confirmar las dos preguntas.
Resultado esperado	Insertado 1, omitido 1, rechazado 1; memoria sin cambios.
Resultado obtenido	Resumen 3 / 1 / 1 / 1. En MySQL aparece RAT-002. En memoria siguen 3 productos.
Estado	**SUPERADA**

Evidencia	`Comprobando MON-003 - Monitor 24 pulgadas...`
	`[ERROR] No existe el proveedor con ID 3. El producto no se puede importar.`
	`Comprobando RAT-002 - Ratón inalámbrico...`
	`[OK] Producto válido para importar.`
	`Comprobando TEC-001 - Teclado mecánico...`
	`[AVISO] El producto ya existe en MySQL. Se omite.`
	`Productos analizados:  3`
	`Productos insertados:  1`
	`Omitidos por código duplicado:  1`
	`Rechazados por datos incorrectos:  1`
	`[OK] Importación finalizada. El inventario en memoria no se ha modificado.`

CP-JDBC-18 · RF-JDBC-07 · Fallo SQL forzado y rollback

Objetivo	Forzar un error a mitad del lote y comprobar que no queda ninguna inserción parcial.
Precondiciones	Memoria con NEW-001 y NEW-002 (fichero JSON de prueba). Ninguno existe en MySQL.
Datos de entrada	Mientras el programa espera la confirmación final, se inserta NEW-002 desde fuera (MySQL).
Pasos ejecutados	1. Importar el JSON. 2. Menú 7 > 8 y confirmar la primera pregunta. 3. Insertar NEW-002 desde el cliente mysql. 4. Confirmar.
Resultado esperado	Error de clave duplicada en NEW-002, rollback y NEW-001 no guardado.
Resultado obtenido	Rollback realizado. En MySQL solo está el NEW-002 insertado desde fuera; NEW-001 no aparece.
Estado	**SUPERADA**

Evidencia	`[OK] Producto válido para importar.  (NEW-001)`
	`[OK] Producto válido para importar.  (NEW-002)`
	`[AVISO] Se van a insertar 2 productos en una única transacción. ¿Continuar? S`
	`[ERROR] Error SQL durante el lote: Ya existe un registro con ese valor (clave duplicada).`
	`[ERROR] Se ha hecho ROLLBACK: no se ha confirmado ninguna inserción del lote (2 productos).`
	`mysql> SELECT codigo,nombre FROM producto WHERE codigo LIKE 'NEW-%';`
	`| NEW-002 | Insertado desde fuera |  (NEW-001 no está)`

CP-JDBC-19 · RF-JDBC-08 · Dos inserciones y commit

Objetivo	Insertar dos categorías de prueba en una transacción y confirmar con commit().
Precondiciones	No existen las categorías 9001 y 9002.
Datos de entrada	Menú 7 > 9. Categorías 9001 y 9002.
Pasos ejecutados	1. Menú 7 > 9 y confirmar. 2. Contestar N a eliminarlas. 3. SELECT en MySQL.
Resultado esperado	Las dos categorías quedan guardadas.
Resultado obtenido	Commit correcto; el SELECT encuentra 9001 y 9002.
Estado	**SUPERADA**

Evidencia	`--- Escenario A: setAutoCommit(false), 2 INSERT y commit() ---`
	`[OK] Commit realizado: el SELECT encuentra las categorías 9001 y 9002.`
	`mysql> SELECT id,nombre FROM categoria;`
	`| 9001 | Demo transacción 9001 |`
	`| 9002 | Demo transacción 9002 |`

CP-JDBC-20 · RF-JDBC-08 · Duplicado y rollback total

Objetivo	Provocar una clave duplicada dentro de la transacción y comprobar que el rollback deshace también la primera inserción.
Precondiciones	No existe la categoría 9003.
Datos de entrada	Escenario B de la misma demostración: categoría 9003 insertada dos veces.
Pasos ejecutados	1. Misma ejecución que CP-JDBC-19. 2. SELECT en MySQL.
Resultado esperado	Error capturado, rollback y la 9003 no existe.
Resultado obtenido	Correcto: la tabla categoria no contiene la 9003. autoCommit restaurado.
Estado	**SUPERADA**

Evidencia	`--- Escenario B: INSERT correcto, INSERT duplicado y rollback() ---`
	`[ERROR] Error esperado capturado: Ya existe un registro con ese valor (clave duplicada).`
	`[OK] Rollback correcto: la primera inserción (categoría 9003) tampoco está en MySQL.`
	`autoCommit restaurado a true y conexiones cerradas.`
	`mysql> SELECT id FROM categoria;  -> 1, 2, 9001, 9002  (no hay 9003)`

CP-SQLITE-01 · Apartado 7 · Primera ejecución de DemoSQLite

Objetivo	Comprobar que se crea el fichero .db y se recuperan las dos filas.
Precondiciones	No existe la carpeta datos/sqlite.
Datos de entrada	Ejecutar la clase DemoSQLite.
Pasos ejecutados	1. Borrar datos/sqlite. 2. Ejecutar DemoSQLite.
Resultado esperado	Se crea el fichero, 2 insertados y se listan SQL-001 y SQL-002.
Resultado obtenido	Correcto.
Estado	**SUPERADA**

Evidencia	`[OK] Conexión SQLite abierta: datos/sqlite/productos_demo.db`
	`[OK] Tabla producto_demo disponible.`
	`Insertados: 2 | Ya existentes: 0`
	`SQL-001 | Producto demo A`
	`SQL-002 | Producto demo B`

CP-SQLITE-02 · Apartado 7 · Segunda ejecución sin duplicados

Objetivo	Comprobar que al repetir la ejecución no se duplican registros.
Precondiciones	Fichero creado en CP-SQLITE-01.
Datos de entrada	Ejecutar DemoSQLite otra vez.
Pasos ejecutados	1. Ejecutar DemoSQLite.
Resultado esperado	0 insertados, 2 ya existentes y solo 2 filas.
Resultado obtenido	Correcto (INSERT OR IGNORE).
Estado	**SUPERADA**

Evidencia	`Insertados: 0 | Ya existentes: 2`
	`SQL-001 | Producto demo A`
	`SQL-002 | Producto demo B`

CP-SQLITE-03 · Apartado 7 · Cierre de recursos y fichero persistente	
Objetivo	Comprobar que los recursos se cierran y el fichero sigue existiendo al terminar.
Precondiciones	Ejecuciones anteriores terminadas.
Datos de entrada	Listar datos/sqlite y comprobar con lsof que ningún proceso tiene el fichero abierto.
Pasos ejecutados	1. ls -l datos/sqlite. 2. lsof sobre el fichero.
Resultado esperado	El fichero existe y no está abierto por ningún proceso.
Resultado obtenido	El fichero existe (12288 bytes) y lsof no muestra procesos. El código usa try-with-resources.
Estado	**SUPERADA**

Evidencia	`[OK] Conexión cerrada. El fichero datos/sqlite/productos_demo.db se conserva ...`
	`$ ls -l datos/sqlite`
	`-rw-r--r--  1 alfonso  staff  12288 Nov  9 18:00 productos_demo.db`
	`$ lsof datos/sqlite/productos_demo.db     (sin resultados)`

3. Incidencias detectadas y correcciones

ID / prueba	Problema detectado	Corrección aplicada	Resultado tras repetir
CP-JDBC-01 (y resto)	Al añadir el driver de SQLite, al conectar aparecía en consola el aviso "SLF4J: Failed to load class org.slf4j.impl.StaticLoggerBinder", que parecía un error.	Se añade la dependencia slf4j-nop 1.7.36 al pom.xml para silenciarlo.	El aviso ya no aparece. SUPERADA.
CP-SQLITE-01/02	Con JDK 25, al ejecutar DemoSQLite con java aparece "WARNING: A restricted method in java.lang.System has been called" (el driver carga su librería nativa).	No es un error y no afecta al resultado. Con JDK 21, que es la versión del proyecto, no aparece. Se documenta en el README.	Salida correcta. SUPERADA.
CP-JDBC-18	Para forzar un fallo SQL a mitad del lote hace falta que el producto pase las comprobaciones previas y luego falle al insertarse.	Antes de la transacción se pide una segunda confirmación; en esa pausa se inserta NEW-002 desde el cliente mysql.	Rollback verificado. SUPERADA.

4. Conclusión de la ejecución

Número de pruebas ejecutadas: **23** Superadas: **23** No superadas: **0** No ejecutadas: **0**

Funcionalidades pendientes y observaciones: no quedan requisitos de la UT2 pendientes. Se han ejecutado CP-JDBC-01 a CP-JDBC-20 y CP-SQLITE-01 a CP-SQLITE-03. Los resultados mostrados por la consola se han comprobado con consultas SELECT en MySQL. Las funcionalidades de la UT1 se conservan. Las categorías de demostración 9001 y 9002 se pueden borrar contestando S al final de la opción 7.9; si se conservan, se borran solas al repetirla.

Fecha: **09/11/2026**